Secure Coding: Threat Modeling

P04:TradeUp

<team member names & ids>

Student ID	Name
26100216	Muhammad rayyan khan
26100200	Muhammad ahmad
26100204	MUHAMMAD SHAHMIR SHER QAZI
26100355	MUHAMMAD UMAR ZUBAIR
25100137	MOHAMMED RAIYAAN JUNAID HAMID

Table of Contents

1. Introduction

The project is a **stock trading simulation platform** that enables hands-on learning for people who want to gain trading experience without the risk of losing money. The platform is intended to provide experiential learning value to users, to supplement their stock trading learning journeys.

Our platform's objectives are to provide a safe and risk-free trading environment that pulls real-time stock market data to simulate the real thing – meaning our environment will capture and simulate the market's movement as it moves in real-time. Furthermore, we aim to provide valuable learning experiences to users with our platform that engage and empower them to improve their stock trading abilities and confidence.

Proposed Feature Set

- Buying/Selling Stocks
- Gamified Leaderboard and Topic-Specific Channels
- AI Insights and Chatbot
- AI News Sentiment Analysis
- Candlestick Charts View
- MarketWatch: to track favorited stocks
- Educational content
- Personalised Notification Systems

2. Instructions

Do the following for each sprint:

- Use **STRIDE** framework to perform threat modeling for each use case of the sprint. Focus only on the use cases that you plan to develop in the sprint.
- Analyze the functionality of each use case to assess the possibility of threats from each aspect of the STRIDE framework, i.e., spoofing, tampering, repudiation, information disclosure, denial of service, and elevation of privilege.
- For each threat, identify the mitigation strategies (i.e., the controls that you will build in your application) to address the threat under consideration.
- Examples are given in this document and slides. Make sure that you strictly follow the pattern given in the examples.
- This document will evolve overtime as threat modeling for use cases of the future sprints are added.
- This document must be submitted along with each of the project deliverables in the future. It must also be kept updated.

3. Threat Modeling: Sprint-1

<Perform STRIDE threat modeling for each use case. You must describe briefly the threat and corresponding mitigation strategies. The following three examples provide a baseline to start with. Make sure that you read and follow the instructions given above.>

Use Case Name	Spoofing	Tampering	Repudiation	Information disclosure	Denial of service	Elevation of Privilege
Update email/password 	An attacker may attempt to log in as the user and update the email/password without authorization.	An attacker may intercept or modify the update request to change email/password to an attacker-controlled	A user may deny that they changed their email/password.	Email update API leaking sensitive user info, or password reset endpoints revealing account existence.	Attackers may flood the email/password update endpoint, preventing legitimate updates.	User may exploit insecure APIs to update another user's email/password.
	Use secure session cookies with HttpOnly, Secure, SameSite flags. Implement rate limiting for login attempts.	Use HTTPS for all communication.	Maintain audit logs for account changes.	Avoid exposing email addresses in responses. Mask sensitive data in logs.	Apply rate limiting per user. Use CAPTCHA when abnormal behavior is detected.	Enforce role-based access control (RBAC).

Add profile picture 	Attacker uploads an image as another user's profile (ID manipulation).	Attacker uploads malicious files. Attacker replaces another user's picture.	User denies uploading a malicious or inappropriate picture.	Uploaded image path may reveal internal file structure. Image metadata may leak GPS/location.	Attacker uploads extremely large files repeatedly.	Attacker may use file upload to execute code or escalate privileges.
	Validate user session ID on server before updating profile.	Validate MIME type and use server-side file type detection (not just extension). Ensure user ownership check on upload.	Keep upload logs (timestamp, user ID, filename).	Strip metadata (EXIF removal). Store images in private bucket or server folder with signed URLs. Do not expose internal storage paths.	Apply maximum upload limits (e.g., 2MB). Use rate limiting on upload endpoints.	Accept only image formats (JPG/PNG/WebP).

PnL Data Visibility to User 	Attacker pretends to be the user to view PnL.	Attacker sends manipulated requests to view another user's PnL.	User disputes viewing or modifying PnL data.	API might leak sensitive trade data.	Attacker might spam the PnL endpoint with heavy queries.	A normal user tries to access admin-level PnL analytics or aggregated data.
	Secure cookies, session timeouts.	Enforce server-side access control: a user can only view their own PnL. Ensure backend recalculates PnL instead of trusting client-sent numbers.	Log PnL query actions (user ID, timestamp).	Encrypt PnL info at rest and in transit.	Rate limiting, caching	Ensure authorization checks on every PnL API endpoint.

Top-Up Wallet with Virtual Currency 	Attacker pretends to be another user to load credits into their wallet.	Client-side requests can be modified to submit higher values, bypass limits, or inject malformed payloads. If you trust the UI, you lose.	User denies topping up or disputes the amount..	Wallet API leaks wallet balance or transaction details of other users.	Attacker spams the top-up endpoint, blocking legitimate transactions.	User tries to exploit admin APIs to modify their wallet balance.
	Server determines wallet ownership using session ID only.	Reject any client-provided monetary values that violate server-defined rules. Perform all amount validation server-side; ignore client-side constraints.	Maintain secure audit logs of wallet transactions. Store timestamp, amount, payment method, and user ID.	Encrypt sensitive data	Enforce rate limiting	Enforce role-based access control (RBAC). Ensure API is secured from loopholes or glitches.

Ai for Stock simulation 	An attacker impersonates the external financial data provider and sends fake market data, causing corrupted predictions.	An attacker modifies the stored ML model file to influence predictions.	A malicious internal user denies triggering a model retraining job that produced incorrect predictions.	Unauthorized users access prediction API logs containing sensitive model parameters or financial data.	Attackers flood the prediction API with high-volume requests, overwhelming resources and stopping normal operations.	A compromised user account gains admin-level access and can alter model configurations.
	Use signed API requests to verify the identity of the data provider before ingesting any data.	Store model artifacts with cryptographic integrity checks and validate signatures before loading the model.	Enable immutable audit logs with timestamps and user IDs.	Apply role-based access control (RBAC) and encrypt all logs both in transit and at rest.	Implement rate limiting on the endpoint to block abusive calls.	Use least-privilege IAM roles and enforce multi-factor authentication (MFA) for all privileged accounts.

View Stock News 	Attacker spoofs the external news provider and sends fake/malicious news to the backend.	Attacker intercepts and alters the news data in transit between backend and external API	External API provider or internal systems could deny sending or modifying news results.	News API could return data containing sensitive or internal information not meant for users.	Attacker spams the news endpoint, causing API rate limits or backend resource exhaustion.	Attacker manipulates the backend request parameters to access data beyond allowed scope
	Store API keys securely on the server-side. never expose to frontend.	Only use HTTPS for external API communication.	Maintain audit logs of all news API calls	Validate returned data before sending to frontend.	Implement rate limiting on news endpoints.	Validate and sanitize all backend parameters before sending to external API.

4. Threat Modeling: Sprint-2

<to be done in the future before Sprint-2 development >

Add sentiment analysis for each news article	Stolen Gemini API key used to generate fake sentiment.	News text altered before analysis	No proof of which article was analyzed	Sensitive data sent to AI provider	Flooding sentiment analysis requests	Unauthorized access to AI orchestration APIs
Threats						
Mitigations	Store API keys in secrets manager and not in env files	Input sanitization and length limits	Log article ID + content hash	Redact sensitive fields before AI calls	Implement rate limiting on the endpoint to block abusive calls.	Role-based access control (RBAC)

local and international news feeds 	Fake local news sources impersonating legitimate publishers	News content modified in transit	News provider denies publishing an article	News API could return data containing sensitive or internal information not meant for users.	Attacker spams the news endpoint, causing API rate limits or backend resource exhaustion.	Attacker manipulates the backend request parameters to access data beyond allowed scope
	API key–based authentication for news providers	Only use HTTPS for external API communication.	Store content hashes for fetched articles	Validate returned data before sending to frontend.	Implement rate limiting on news endpoints.	Validate and sanitize all backend parameters before sending to external API.

User Search & Add Friends 	Fake user profiles impersonating real users	Manipulation of friendship relationships in database	Users denying sending or accepting friend requests	User enumeration via search (email)	Search endpoint abuse	Accessing admin-only social graph APIs
	Strong authentication and session expiration	Authorization checks on every request	Log sender ID, receiver ID, timestamp, action	Limit searchable fields (no email/phone)	Implement rate limiting on the endpoint to block abusive calls.	RBAC for social graph management

View Other Users' Stock Portfolios 	Attacker impersonates another user to view restricted portfolios	Portfolio data modified during transit	Users deny viewing another's portfolio	Unauthorized access to private portfolios	Excessive portfolio view requests	Bypassing privacy controls to view restricted portfolios
	Strong authentication (JWT)	HTTPS for all communications	Log viewer ID, portfolio owner ID, timestamp	Privacy settings (public / friends / private)	Implement rate limiting.	Strict RBAC and permission checks

AI Stock Simulation Model 	Fake or malicious data sources posing as legitimate PSX data	Unauthorized modification of trained model weights	Inability to prove model configuration or parameters	Exposure of internal model logic or parameters	Large or malformed datasets causing crashes	Unauthorized users modifying training configurations
	Whitelist trusted PSX data providers	Access control on model artifacts	Store model metadata	Restrict access to training data and models	Enforce maximum file size, row count, and feature count before ingestion	Role-based access control (RBAC)

5. Threat Modeling: Sprint-3

<to be done in the future before Sprint-3 development >

Reactions / Likes / Upvotes Threats	Attacker impersonates another user to like/react to posts they didn't create.	Reactions are changed, removed, or added without authorization (counts or types).	User denies reacting ("I never liked that post").	Reactions intended to be private are visible to others.	Flooding a post with reactions to misrepresent engagement or degrade performance.	User manipulates reactions globally across posts they don't own.
Mitigations	Strong authentication, session protection, MFA, account verification	Server-side validation, integrity checks, audit logs	Timestamps, immutable activity logs, signed actions	Access control, privacy settings enforcement	Rate limiting, bot detection, CAPTCHA	Role-based access control (RBAC)

Block/Mute users 	An attacker impersonates a user and blocks or unblocks someone without the real user's intent.	An attacker alters mute/block settings	A user denies muting or blocking another user	A blocked user learns they were blocked (or muted), enabling harassment or retaliation.	An attacker blocks many users on behalf of a victim, cutting off communication.	A blocked user bypasses restrictions
	Strong authentication, session protection	Server-side authorization checks	Audit logs	Do not notify blocked users, return generic errors ("message not delivered")	Implement rate limiting	Centralized block enforcement across all messaging paths, consistent access control checks

Community posts 	An attacker impersonates another user and creates posts or comments under their identity.	An attacker edits, deletes, or modifies posts/comments without authorization.	A user denies creating a post or comment	Private or restricted posts are visible to unauthorized users	An attacker floods the community with posts/comments, reducing usability and trust.	A normal user gains moderator abilities
	Strong authentication, session management, account verification indicators	Server-side authorization	Audit logs	Access control checks	Implement rate limiting.	RBAC for social graph management

Chart Studying tools 	Malicious actor injects false market data into the client-side chart feed, misleading the user's analysis.	Attacker modifies client-side JavaScript to alter how technical indicators (e.g., PnL) calculated/displayed.	User claims they did not delete a complex study layout or custom indicator set.	Sharing features (e.g., "Share Chart Image") inadvertently include PII or account position data in the generated artifact.	User or attacker loads excessive indicators or historical data points (e.g., 10k candles), crashing the browser or straining the API.	Attacker injects malicious scripts into "Note" or "Label" tools. If the chart is shared, the script executes in the victim's session.
	Implement mutual TLS (mTLS) and digital signatures for data streams to verify origin integrity.	Use subresource integrity (SRI) hashing for script loading; Obfuscate critical logic; validate data integrity on periodic server syncs.	Enable comprehensive audit logging for all CRUD operations on user configurations (Creation, Deletion, Modification).	Force-disable account overlays (balance, position size) during image generation/export; apply strictly scoped permissions for shared links.	Enforce strict pagination limits; Implement client-side rate limiting for rendering requests; Set hard caps on active indicators per chart.	Rigorous input sanitization (e.g., DOMPurify) on all user-generated text fields; Implement strict Content Security Policy (CSP) blocking inline scripts.

Trading from Chart view 	Attacker forces the user's browser to execute a trade (Buy/Sell) without their consent via a forged request from a different context.	Attacker intercepts the API request generated by a UI action (e.g., dragging a limit line) and modifies the price or quantity before it reaches the server.	User claims a trade executed via a chart click was accidental or system-generated.	The API response for the chart overlay includes full account details (balance, full portfolio) unnecessarily, exposing it to interception or logs.	User or script rapidly clicks "Buy/Sell" or drags order lines, overwhelming the order gateway.	A user with "View Only" permissions (e.g., a sub-account or shared view) manages to execute a trade by manually calling the API endpoint.
	Implement strict Anti-CSRF tokens on all order endpoints; Enforce SameSite=Strict cookies; Require re-authentication (2FA/PIN) for high-value trades.	Server-side validation of all order parameters against current market constraints; Digitally sign critical payload data at the client side if high security is required.	Transactional Logging: Record precise timestamp, UI element ID, mouse coordinates (optional but helpful context), and server-side receipt ID; Require User Confirmation (e.g., "Confirm Buy") for every action.	Data Minimization: API should return only the specific order/position data needed for the visualization (e.g., `position_id`, `entry_price`), not the full account profile.	Rate Limiting: Enforce strict per-second request limits on order endpoints; Implement Debouncing on UI elements (prevent double-clicks); Use Idempotency Keys to prevent duplicate execution of the same click.	Role-Based Access Control (RBAC): Strictly enforce execution permissions on the backend API, regardless of UI state; Verify user holds "Trader" role before processing *any* `POST /orders` request.

AI game mode based on market simulation 	An attacker could spoof the **Architect Agent's** identity to inject malicious market trajectories into the cached library, forcing unrealistic price drops for certain users.	A user might intercept and modify the `trajectoryJson` response to manually inflate their synthetic balance or give themselves "insider knowledge" of upcoming ticks.	An experienced trader might deny executing a specific loss-making trade during the "30-Day Oracle" session, claiming the AI "manipulated" the UI.	The "Realistic Outlook" preset might accidentally leak proprietary market sentiment or unannounced PSX news if the **Chronicler Agent** is poorly prompted.	Attackers could spam the "Realistic Outlook" refresh, forcing massive Gemini API costs or locking the `SimulationState` table for other users.	A user could manipulate the `presetType` parameter to access "Pro" realistic outlooks without reaching the
	Use API Key rotation and strictly scoped access for the **Gemini 1.5 Flash** service. Sign generated JSON blobs to ensure they originated from the backend.	Encrypt the `trajectoryJson` stored in PostgreSQL. Validate the client-side price updates against the server-side source of truth during every "Buy/Sell" transaction.	Maintain immutable **Transaction Logs** and **Audit Logs** that link specific trades to a unique `SimulationStateId` and timestamp.	Implement **Prompt Scrubbing** to remove sensitive real-world PII before context injection. Mask exact AI confidence scores in the public-facing UI.	Implement **Global Rate Limiting** for Oracle generation. Serve the cached `SimulationState` to all users once per 24-hour cycle to minimize LLM hits.	Enforce **Role-Based Access Control (RBAC)** on the `/oracle/generate` endpoint. Verify the user's `Role` (e.g., TRADER vs. PRO) before executing LLM calls.

Comments / Replies Threats	Attacker posts a comment pretending to be someone else.	Unauthorized editing or deletion of comments.	User denies posting a comment.	Comments meant for restricted visibility are exposed publicly.	Flooding posts with comments to disrupt conversation or degrade platform.	User manipulates reactions globally across posts they don't own.
Mitigations	Strong authentication, session protection, MFA, account verification	Server-side validation, integrity checks, audit logs	Timestamps, immutable activity logs, signed actions	Access control, privacy settings enforcement	Normal users gain privileges to delete or pin comments they shouldn't control.	Role-based access control (RBAC)

Natural language order ticket transaction feedback Threats	Attacker injects a fraudulent WebSocket message to confirm a fake trade.	Man-in-the-Middle (MITM) alters the feedback string (e.g., changing "10 shares" to "100 shares") to mislead the user during the order flow.	User claims the feedback displayed different values than the final trade execution.	Feedback contains sensitive data (Account ID, Portfolio Value) leaked via client-side logs or browser history.	Flooding the feedback channel to delay or hide actual trade updates, preventing timely user intervention.	The feedback generator service fetches data for an unauthorized `user_id` due to insecure direct object references (IDOR).
Mitigations	Implement Cryptographic Session Binding; Use Secure WebSockets (WSS) with Origin validation.	Message Integrity Checks (HMAC); End-to-end encryption (TLS 1.3); Client-side checksum verification of the feedback payload.	Log the exact feedback string sent to the client with a unique Transaction ID; Implement a client-side "Receipt" acknowledgment.	Exclude PII from feedback strings; Use transient state management (do not persist feedback in local storage/logs).	Implement Rate Limiting on feedback broadcasts; Prioritize "Order Confirmation" packets over general feedback updates.	Enforce strict Server-side Session Authorization; Ensure the feedback engine only has "Read" access to the current user's active session data.

Realistic delay simulation of order transaction 	An attacker spoofs the source of the simulated delay data to inject unrealistic, exploitable delay times (e.g., zero delay) for their orders.	An attacker intercepts and modifies the simulated delay duration or the order status (e.g., changing a pending order to executed prematurely) while the order is in the simulated delay state.	A user denies that a specific simulated delay was applied to their trade, potentially to dispute an unfavorable execution price that resulted from the delay.	The simulation service exposes internal configuration parameters (e.g., delay calculation algorithm, latency simulation settings) or simulation variables in logs or error messages.	An attacker rapidly submits a large volume of orders that rely on the delay simulation, exhausting the resources of the simulation engine or the queue handling delayed orders.	A user bypasses authorization to manually set the simulated delay time to zero, or access configuration APIs to alter the global delay parameters.
	Use server-side validation and strong authentication for internal services generating the delay data. Implement cryptographic integrity checks on the simulation data.	Ensure that the order state is immutable during the simulation delay period. Use HTTPS/TLS for all communication.	Implement comprehensive, immutable server-side logs recording the start time, end time, and specific delay duration applied to every transaction (Timestamping, immutable activity logs).	Mask or omit sensitive configuration/user data in logs. Return generic error messages to the client (Mask logs).	Implement rate limiting on order submission endpoints per user. Enforce a cap on the number of simultaneous pending (delayed) orders per user.	Enforce strict Role-Based Access Control (RBAC). Ensure the server-side logic cannot be overridden by client-side input for delay parameters.

Visualize portfolio data with graphs and charts 	Attacker pretends to be the user to view their portfolio data (e.g., PnL, holdings).	Attacker intercepts and modifies the portfolio data returned to the client, displaying incorrect PnL or asset values on the charts.	User denies viewing a specific chart or a portfolio state on a certain date, possibly to dispute a trading decision.	API leaks unauthorized private portfolio data (e.g., PnL, holdings) of other users.	Attacker rapidly requests complex portfolio visualizations (e.g., historical data spanning years) to exhaust backend resources.	A normal user gains unauthorized access to admin-level aggregated portfolio data or configuration settings for charts.
	Strong authentication (JWT, MFA), secure cookies, and session timeouts.	Enforce HTTPS/TLS for all communication. Validate data integrity server-side, ensuring client data matches the source of truth.	Log all portfolio data access and visualization actions (user ID, timestamp, data queried). Maintain immutable audit logs.	Enforce privacy settings (public/friends/private). Use Data Minimization to return only necessary data for the current user's view.	Implement rate limiting on the visualization API endpoint per user. Enforce limits on the volume of historical data returned.	Enforce strict Role-Based Access Control (RBAC). Verify user permissions on the server-side for all data access requests.

One-sentence trade journaling ([Target Hit], [Panic/Emotion], or [Needed Cash])	Attacker spoofs a journal entry, attributing a trade reason (e.g., Target Hit) to a user who did not create it.	An attacker intercepts the request and modifies the reason string (e.g., changing "Target Hit" to "Panic/Emotion") or alters the trade ID associated with the entry.	A user denies writing a specific emotional journal entry (e.g., [Panic/Emotion]) tied to a trade, to protect their trading image.	The journal entry API leaks personal trading details (e.g., full trade history, PnL) when querying a user's one-sentence journal entries.	Attacker spams the journal creation endpoint with numerous short entries, causing database bloat or slowing down the user's trade history view.	A normal user exploits a flaw to modify another user's journal entry, or access restricted "private" journal settings.
Threats						
Mitigations	Verify user identity via session token/JWT on the server-side before accepting the journal entry.	Validate the length and content of the journal string to prevent injection. Ensure the journal is immutably linked to the trade ID.	Use immutable audit logs to record the user ID, trade ID, and exact content of the journal entry upon creation.	Enforce strict server-side authorization: a user can only read/write their own journal entries. Do not return PII in the API response.	Implement rate limiting on the journal creation endpoint per user. Implement a length limit on the journal string (e.g., 100 characters).	Enforce strict Role-Based Access Control (RBAC) to ensure a user can only perform CRUD operations on their own journal entries.

Save community posts Threats	Attacker spoofs the user's ID to save a post without their consent.	Attacker modifies the saved post metadata (e.g., timestamp) in the user's saved list database.	User denies saving a specific community post, possibly after the content becomes controversial.	API leaks the user's full list of saved posts to an unauthorized third party.	Attacker rapidly saves a massive number of posts to a user's account, consuming excessive database resources.	A normal user exploits a vulnerability to save a post to a different user's restricted collection.
Mitigations	Verify user identity on the server-side before performing the "save" action, strictly checking the session ID/token against the request ID.	Ensure the saved post relationship data (user ID, post ID, save time) is immutable after creation. Use HTTPS/TLS for all save/retrieve requests.	Implement immutable audit logs recording the user ID, post ID, and exact timestamp when the save action occurred.	Enforce strict server-side authorization checks (user can only retrieve their own saved list).	Implement hard limits on the total number of posts a user can save. Implement rate limiting on the "Save Post" API endpoint per user.	Enforce strict Role-Based Access Control (RBAC) on the 'saved' collection API. Ensure all database queries for saved posts are filtered by the authenticated user's ID.

Mention users in posts/comments Threats	Attacker posts a comment pretending to be someone else.	Unauthorized editing or deletion of comments.	User denies posting a comment.	Comments meant for restricted visibility are exposed publicly.	Flooding posts with comments to disrupt conversation or degrade platform.	User manipulates reactions globally across posts they don't own.
Mitigations	Strong authentication, session protection, MFA, account verification	Server-side validation, integrity checks, audit logs	Timestamps, immutable activity logs, signed actions	Access control, privacy settings enforcement	Normal users gain privileges to delete or pin comments they shouldn't control.	Role-based access control (RBAC)

Post chart and images in posts Threats	An attacker uploads a malicious or misleading image/chart pretending to represent legitimate market data or a trusted source.	An attacker uploads a file that contains malware or a malicious script hidden within image metadata (e.g., EXIF data).	A user denies posting a malicious, copyrighted, or inappropriate image/chart.	An uploaded chart screenshot or image accidentally includes PII or sensitive trade position details.	An attacker uploads extremely large files repeatedly, consuming excessive storage or processing resources (image resizing).	An attacker exploits a vulnerability in the image processing library to gain remote code execution on the server.
Mitigations	Validate the user session ID and source of chart data before allowing post.	Use server-side file type detection. Strip all malicious or unnecessary metadata (EXIF). Store files with obfuscated names.	Keep detailed, immutable logs recording the user ID, file hash, and timestamp for every post/upload action.	Automatically strip image metadata (EXIF). Force-disable sensitive overlays (like PnL/balance) during chart image generation for sharing.	Enforce strict maximum file size limits (e.g., 5MB). Implement rate limiting on the file upload endpoint per user.	Use least-privilege principles and sandboxing for the file processing service. Use hardened and updated image libraries.

6. Threat Modeling: Sprint-4

<to be done in the future before Sprint-4 development >

7. Common Threats and Mitigations Strategies/Controls

<Here are common threats and mitigation strategies (controls that you can build to address different threats). This is not an exhaustive list but will be useful when you analyze system use cases for threat modeling. Use this just as a guideline, you are NOT required to implement each control in your system, i.e., implement only those which are necessary and relevant for your system.>

Authentication & Authorization Threats

Threat	Description	Mitigation Strategies/Controls
Weak Passwords and Credential Stuffing	Attackers reuse or guess weak passwords. Credential stuffing happens when attackers use leaked username-password pairs from one service to gain unauthorized access to another where users have reused credentials.	● Enforce strong password policy ● Implement Multifactor authentication (MFA) ● Limit login attempts ● Use CAPTCHA ● Enable account lockout after a specific number of failed attempts.

Session Hijacking	Stealing or reusing session tokens.
	Session hijacking (aka *session takeover*) occurs when an attacker steals or predicts a user's active session identifier (session ID, cookie etc.) and uses it to take over that user's authenticated session—effectively impersonating them without knowing their password.
		● Use HTTPS to avoid sniffing/eavesdropping,
		● Secure cookies
		  o HttpOnly (Makes cookie inaccessible to JavaScript),
		  o Secure (cookie is sent only over HTTPS),
		  o SameSite (*strict*: cookie is only sent if the request originates from the same site)
		● Regenerate session IDs after login,
		● Short session lifetimes (session expiry).
		● Implement MFA.

Token Theft (JWT / OAuth)	Token stolen from insecure storage or during transmission. An attacker steals or obtains authentication tokens used to prove a user's identity or access rights, allowing them to impersonate that user or access protected resources without needing their password.	● Store tokens securely (Keychain/Keystore), ● Use short-lived tokens, ● Rotate refresh tokens, ● Sign tokens.
Privilege Escalation	Regular user gains admin-level access.	● Implement strict RBAC (Role-Based Access Control)

		● Implement ABAC (Attribute-Based Access Control) ● Perform server-side authorization checks ● Validate user roles in backend logic
Broken Access Control	User accesses data or functionality they shouldn't. Broken access control usually occurs due to mistakes like unprotected endpoints, excessive trust in client-side checks, insecure direct object reference (IDOR).	● Enforce server-side authorization checks, ● Never rely solely on client-side validation, ● Use secure API gateways.

Input Validation & Injection Threats

Threat	Description	Mitigation Strategies/Controls
SQL Injection	Attacker manipulates database queries via unvalidated user input.	● Use parameterized queries, ● Perform input validation, ● Enforce least privilege on database accounts.
XSS (Cross-Site Scripting)	Attacker injects malicious scripts into web pages.	● Sanitize input ● Encode output ● Use Content Security Policy (CSP).
Command Injection	User input triggers system commands.	● Validate and sanitize input, ● Use safe APIs (avoid exec()), ● Escape shell metacharacters.
Insecure Deserialization	Attacker alters serialized objects to execute code.	● Validate input types, ● Sign or encrypt serialized objects, ● Avoid deserialization of untrusted data.
Unvalidated Redirects/Forwards	Attacker redirects users to malicious sites.	● Validate redirect targets, ● Use whitelists,

		● Avoid user-controlled URLs.
XML External Entity (XXE)	An XXE attack happens when an XML parser processes malicious XML that defines external entities, which can then be used to read local files, perform network requests, or crash the system. It's a way for attackers to abuse how XML parsers handle external resources.	● Validate and sanitize XML input ● Use less risky data formats, if possible, such as JSON or other formats that don't have external entity features.
Input Overflow / Buffer Overflow	Input exceeds expected length, causing memory corruption.	● Enforce strict length checks and bounds validation.

Data Protection & Cryptographic Threats

Threat	Description	Mitigation Strategies/Controls
Data Leakage	Sensitive data exposed in logs, caches, or storage.	● Mask logs, ● Disable caching for sensitive info, ● Secure storage (Keychain/Keystore), ● Data minimization.
Insecure Data Storage	Sensitive data stored or transmitted in plaintext (unencrypted).	● Encrypt data at rest (AES-256), ● Use OS-provided secure storage, ● Avoid storing plaintext credentials.
Hardcoded Secrets / API Keys	API keys or credentials hardcoded in code or configuration files.	● Use environment variables, ● Secure vaults (e.g., AWS Secrets Manager), ● Avoid storing secrets in source code.
Man-in-the-Middle (MITM)	Attacker intercepts communication.	● Enforce HTTPS, ● Verify SSL certificates.

Application Logic Threats

Threat	Description	Mitigation Strategies/Controls
Cross-Site Request Forgery (CSRF)	Attacker tricks user into executing unwanted actions.	● Use anti-CSRF tokens, ● SameSite cookies, ● Verify request origins.
Business Logic Abuse	Exploiting flaws in process flow (e.g., skipping payment).	● Validate workflow on server-side, ● Enforce transaction integrity checks.
Server-Side Request Forgery (SSRF)	An attacker tricks a server into making HTTP requests to internal or external resources that the attacker normally wouldn't be able to access.	● Validate and whitelist URLs ● Block internal/private IP ranges ● Sanitize and canonicalize input

Miscellaneous Threats

Threat	Description	Mitigation Strategies/Controls
Clipboard / Screenshot Leakage	Sensitive info copied or visible in screenshots.	● Avoid copying sensitive data, ● Disable screenshots for sensitive screens.
Insecure Third-Party SDKs	Vulnerabilities or tracking from libraries.	● Vet SDKs for security, ● Limit permissions, ● Use trusted sources. ● Don't use outdated libraries
Insecure CORS Configuration	Overly permissive cross-origin access.	● Restrict origins, ● Disallow *, ● Validate allowed domains.

Sensitive Data in URL	Data exposed in logs or browser history.	• Use POST for sensitive requests, avoid query parameters for secrets.
Insecure File Uploads	Uploads allow malicious code execution.	• Validate file types, scan for malware, store outside web root.
API Security	Excessive or automated requests.	• Implement authentication and authorization for all endpoints. • Validate and sanitize all input data (including JSON, XML). • Implement rate limiting and throttling to prevent abuse. • Avoid exposing internal object references (IDs, filenames). • Use API gateways or WAFs for additional protection. • Ensure proper CORS configuration — only allow trusted domains. • API keys. •
Improper Exception Handling	Crashes or unhandled exceptions reveal internals.	• Catch and handle exceptions gracefully •
Sensitive data in logs	Logging passwords, tokens	• Mask or omit sensitive data in logs.
Information Disclosure	Revealing system details in error messages.	• Return generic errors to users • log detailed errors securely.

8. STRIDE Framework and OWASP Top 10

There is **NO strict mapping** between STRIDE and OWASP. However, If you use STRIDE thoroughly to identify threat types while keeping OWASP top 10 in mind, they provide complementary coverage of threats and vulnerabilities. The following table shows an approximate mapping.

STRIDE CATEGORY	OWASP Top 10
SPOOFING Impersonating a user, system, or service	A07 – Identification & Authentication Failures A01 – Broken Access Control
TAMPERING Unauthorized modification of data or requests	A01 – Broken Access Control A03 – Injection (SQLi, XSS, etc.) A08 – Software & Data Integrity Failures
REPUDIATION Lack of proper logging or audit trails	A09 – Security Logging & Monitoring Failures A04 – Insecure Design
INFORMATION DISCLOSURE Exposure of sensitive information	A02 – Cryptographic Failures A04 – Insecure Design A05 – Security Misconfiguration
DENIAL OF SERVICE Degrading or blocking system availability	A06 – Vulnerable & Outdated Components A10 – Server-Side Request Forgery (SSRF)
ELEVATION OF PRIVILEGE Gaining privileges beyond authorization	A01 – Broken Access Control A04 – Insecure Design

9. Security Engineer

< Each team must designate one member as the **Security Engineer**. While the entire team is responsible for implementation of the project's security features, the Security Engineer will take the lead in overseeing and ensuring the overall security of the project. **>**

Name of the Security Engineer	Umar Zubair

10. Use of Generative AI

GenAI was used to develop the STRIDE points for our use cases to, to provide a comprehensive review of our use cases.

11. Who Did What?

Name of the Team Member	Use Cases
Muhammad Rayyan Khan	Save posts, mention users in posts or comments, post images and charts
Muhammad Ahmad	
Muhammad Umar Zubair	Ai stock simulator chatbot + game, Neo-brutalism UI overhaul
Mohammad Raiyaan Junaid Hamid	NLP order ticket, Realistic transaction delay simulation, Visualised Portfolio Data using graphs and charts, One-sentence trade journaling, Reset portfolio (w/history log)
Muhammad Shamir Sher Qazi	Ai stock simulator chatbot

12. Review checklist

Before submission of this deliverable, the team must perform an internal review. Each team member will review one or more sections of the deliverable.

Section Title	Reviewer Name(s)
Threat modeling sprint-4	M. Raiyaan Junaid Hamid